

INFORMÁTICA

[Área personal](#) / [Mis cursos](#) / [\(29185\) INFORMÁTICA](#) / [Tema 8. Programación dinámica](#) / [Lab 8. Programación dinámica](#)

Lab 8. Programación dinámica

Resuelve todos los ejercicios en la misma entrega. Esta página realizará unas pruebas básicas y no permitirá entregas con errores sintácticos o con demasiados fallos de funcionamiento.

1. Subsecuencia común más larga

Implementa la función `lcs(a,b)` que devuelve la subcadena común más larga entre las cadenas `a` y `b`. Utiliza un algoritmo de programación dinámica.

2. Camino más corto

Un algoritmo de programación dinámica interesante es el *Algoritmo de Floyd*, que calcula la distancia más corta entre dos puntos. Busca información sobre este algoritmo (por ejemplo, en [esta página](#)) e impleméntalo en Python.

Implementa la función `distancia(i,j,G)` donde `i` y `j` son números de nodo, `G` es el grafo, representado como una secuencia de arcos. Cada arco se representa como una tupla `(coste, a, b)` donde `coste` es un número entero positivo que representa el coste del arco, y `a,b` son los números que identifican a los nodos origen y destino respectivamente. Puede asumirse que el grafo tiene todos los nodos entre `0` y un valor máximo `m`.

2.1. Recurrencia

Sea $D_k(i,j)$ el camino más corto de `i` a `j` usando solo los `k` primeros vértices del grafo como puntos intermedios.

$$D_k(i,j) = \min\{D_{k-1}(i,j), D_{k-1}(i,k) + D_{k-1}(k,j)\}$$

Caso base: $D_0(i,j) = \text{coste}_{ij}$

3. Distancia de Levenshtein

La distancia de Levenshtein o distancia de edición mide la diferencia entre dos cadenas de texto como el número mínimo de operaciones de edición que hay que realizar para convertir una cadena en otra. Busca información sobre este algoritmo (por ejemplo, en [esta página](#)) e impleméntalo en Python.

Implementa la función `dist_lev(s,t)` que devuelve la distancia de Levenshtein entre la cadena `s` y la cadena `t`.

3.1. Recurrencia

$$d(i,j) = d(i-1,j-1) \text{ si } s[i] = t[j]$$

$$d(i,j) = 1 + \min\{d(i-1,j), d(i,j-1), d(i-1,j-1)\} \text{ si } s[i] \neq t[j]$$

Casos:

- Mismo carácter: $d(i-1,j-1)$
- Inserción: $1 + d(i-1,j)$
- Borrado: $1 + d(i,j-1)$
- Modificación: $1 + d(i-1,j-1)$

Pruebas

A continuación se incluyen las 3 pruebas que será necesario pasar para aceptar esta entrega. Consulta [esta página](#) para aprender cómo pueden ayudarte las pruebas a desarrollar tu propio programa.

```
# Descomenta la siguiente línea y la última para ejecutar las pruebas
# from unittest import TestCase, main

class Test(TestCase):
    def test_lcs(self):
        casos = [
            ('', '', ''),
            ('abcde', 'cde', 'cde'),
            ('abcde', 'aBcDe', 'ace'),
            ('ababcbcd', 'abbccdde', 'abbccde')
        ]
        for a,b,c in casos:
            self.assertEqual(lcs(a,b),c)

    def test_distancia(self):
        G = ((3,0,1),(5,0,2),(1,0,3),(7,2,3),(7,2,4),(1,2,5),(4,3,5),
            (3,1,0),(5,2,0),(1,3,0),(7,3,2),(7,4,2),(1,5,2),(4,5,3))
        casos = [(G,3,4,12),(G,2,4,7),(G,2,3,5),(G,0,2,5)]
        for G,i,j,d in casos:
            self.assertEqual(distancia(i,j,G), d)

    def test_dist_lev(self):
        casos = [
            ('abcde', 'acdf', 2),
            ('abcde', 'adfg', 4),
            ('abcd', 'f', 4),
            ('', 'abc', 3)
        ]
        for s,t,d in casos:
            self.assertEqual(dist_lev(s,t), d, 'd({},{}) debe ser {}'.format(s,t,d))

# Si usas Jupyter o VSCode descomenta la última línea
# Si usas IDLE, Python o PyCharm descomenta la penultima
# main()
# main(argv=['first-arg-is-ignored'], exit=False)
```

Estado de la entrega

Número del intento	Este es el intento 1.
Estado de la entrega	No entregado
Estado de la calificación	Sin calificar
Fecha de entrega	sábado, 15 de junio de 2019, 00:00:00
Tiempo restante	141 días 10 horas
Última modificación	-

Agregar entrega

Todavía no has realizado una entrega

◀ Tema 8. Programación dinámica

Ir a...

Tema 9. Métodos estocásticos ▶



